Cluster Computing (2025) 28:154
https://doi.org/10.1007/s10586-024-04854-0

Intelligent cache prefetchers in HPC architecture

Farah Fargo[1] · Mitchell Diamond[1] · Olivier Franza[1] · Paul Foose[1] · Jack Adiletta[1] · Matthew Adiletta[1] · Simon C. Steely Jr[1]

Received: 28 June 2024 / Revised: 2 October 2024 / Accepted: 22 October 2024 / Published online: 21 January 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
Reinforcement learning (RL) is revolutionizing the field of Artificial Intelligence (AI) and represents a step ahead towards building an optimal and autonomous system with a higher level of understanding (Arulkumaran et al. in IEEE Signal Processing Mag 34(6):26–38, 2017). One of the main goals for AI is to produce fully autonomous agents to interact with several features and learn the optimal behavior to optimize. Applications vary in data access patterns and a static hardware configuration is not idea for all phases of a workload. Today Xeon cores have multiple data prefetchers which fetch the next sets of data to be used, however, there are problems with these prefetchers as they may interact in destructive ways. This destructive behavior can cause several problems such as an increase in cache pollution, bottlenecks in the memory bandwidth, and additional occupancy to critical path demand queues. Managing the aggressiveness of the prefetchers are necessary to mitigate these problems. Current hardware prefetchers manage the aggressiveness of prefetchers by monitoring telemetry such as memory bandwidth and accuracy. However, there are problems with this approach as the telemetry data does not necessarily correlate with the overall system performance. In addition, other solutions show optimizing prefetchers individually to manage the system performance rather than allowing multiple features to work together. This research introduces hierarchical smart agents using reinforcement learning to find the optimal aggressiveness for the MLC prefetchers on runtime managed by the Smart Prefetchers Manager (SPM). We have expanded our previous work and evaluated more workloads on a hierarchical model and applied reinforcement learning in addition to offline training approach. This approach is implemented and evaluated on single core, single process environment to optimize the three Mid-level Cache (MLC) prefetchers on run time. Results demonstrated that using the reinforcement learning can optimize up to 7.18% improvement in instructions per cycle (IPC) over the state-of-the-art hardware solution.

Keywords Cache prefetching · Reinforcement learning · Q-learning · Performance optimization · HPC system

[1] Intel Corporation, 75 Reeds Rd, Hudson, MA 01749, USA

✉ Farah Fargo
farah.e.fargo@intel.com

Mitchell Diamond
mitchell.diamond@intel.com

Olivier Franza
olivier.franza@intel.com

Paul Foose
paul.e.foose@intel.com

Jack Adiletta
jack.adiletta@intel.com

Matthew Adiletta
matthew1.adiletta@intel.com

Simon C. Steely Jr
simon.c.steely.jr@intel.com

1 Introduction

With the increasing complexity of algorithms and large volume of data requiring more resources and computations to train, the demand of using AI and High-Performance Computing (HPC) to optimize the capabilities of modern computing systems such as efficiency, accuracy, and scalability has been increasing rapidly [1]. Caches are hardware structures used to reduce memory access latencies by bringing smaller amounts of memory closer to the core [2]. Cache prefetching is a technique to proactively optimize performance by fetching instructions and moving data before its requested by the CPU. This is used to reduce latency by speculating about future memory accesses and populating the caches with this information. It focuses on using spatial and temporal locality in memory access patterns. Speculative analysis using spatial locality and memory access patterns help prefetchers decide what information might be valuable. Prefetching improves the standalone performance of many applications and is currently used in almost all commercial processors [3–7]. The benefits of cache prefetching are reducing or eliminating cache misses, diluting the bottleneck caused by a memory bound workload and optimizing instructions execution. However, the challenges with prefetching may have significant negative consequences on performance. This includes increasing cache misses by evicting useful cache lines with useless cache lines; high volume speculative requests for both necessary and unnecessary cache blocks may bottleneck memory bandwidth and significantly increase latency on critical path memory requests; prefetching increases pressure on demand queues and pipeline stages, adding latency to hardware resources [8]. Ideally, prefetchers should include optimal features by being accurate, timely, and complete. For accurate, it means that the prefetcher only requests cache blocks that will be needed by the demand stream. For timely, it means that the prefetcher is ahead of the demand memory access stream such that all prefetched blocks are available in the cache before they are needed, but not too early such that the prefetched data is evicted before the demand request arrives. For complete, it means that the prefetcher requests all future memory accesses needed in the demand stream [9, 10]. In addition, different types of prefetchers are used to improve completeness by capturing various types of locality and memory access patterns.

Today, AI is being applied to solve complex problems for performance optimization in a wide variety of applications on HPC systems [1]. Several applications that we are using daily use AI algorithms. For example, Google search engine is using a machine learning algorithm to search the user requests [11]. In addition to that, the most effective way to solve task is to use AI as it learns, develop, predict, and improve the quality of data usage [12]. The work described in this paper addresses aspects of accuracy and timeliness by managing the aggressiveness of a prefetcher using the AI capabilities. Managing the aggressiveness has a significant impact on performance; however, existing hardware solutions are insufficient, and solutions presented in literature leave unaddressed problems:

•Deep/Machine learning-enabled solutions require pretraining on a target set of workloads to appropriately configure the prefetchers [13–15].

•Simple hill-climbing solutions converge slowly on optimal prefetcher configurations in dynamic environments and are prone to becoming "stuck" in a single configuration that is not necessarily globally optimal [16–18].

•Several solutions presented in literature do not use a fair reward scheme to maximize overall system performance.

This paper considered a single core, single application system with multiple active data prefetchers which resulted in demonstrating significant performance improvement over baseline with minimal hardware overhead. The goal is to show the importance of artificial intelligence across cloud, HPC, and SPEC CPU2017 workloads at the hardware level and realize the significant performance improvements.

1.1 Contribution

To address the existing prefetchers problems we introduce Smart Prefetcher Manager (SPM) architecture employing reinforcement learning to find the optimal prefetcher aggressiveness policy for multiple prefetchers at runtime. SPM agent adjusts prefetcher configurations to optimize a Quality of Service (QoS) metric representing the overall system performance. It optimizes synergism between prefetchers which maximizes the overall system performance better than an individual prefetcher's performance.

•As a motivation for our study, we have demonstrated prefetchers toggling technique to show preliminary results for managing the prefetchers and supported by results detailed in methodology section.

•Smart Prefetcher Manager (SPM) Architecture is proposed, hierarchical reinforcement learning agents employing Q-learning used to manage the prefetcher aggressiveness on the Middle Level Cache (MLC) prefetchers at runtime.

•Details of the optimization techniques used to improve the performance of SPM agent, with an impact study using workloads from the HPC, Cloud and SPEC CPU2017 suite workloads.

We have expanded our previous work [2] by enabling a bidirectional hierarchical model to manage the three MLC

prefetchers (ACL, DPL, and AMP), enabling and evaluating various configurations to find the optimal configurations to be used for IPC optimization in SPM architecture. We have expanded MLC knobs to be monitored and included additional telemetries that have an impact on the performance. In addition to that, we have expanded workload evaluation to include various HPC, Cloud, and CPU SPEC17 workload traces and analyze the impact compared to baseline.

2 Background

This research targets the Mid-Level Cache (MLC) prefetchers and manages the prefetchers aggressiveness using machine learning capabilities. In this section, three existing MLC prefetchers are described and the importance of prefetchers aggressiveness are discussed.

2.1 MLC prefetchers

The Middle Level Cache (MLC) for an Intel Xeon core contains three prefetchers: the Adaptive Multipath Probability (AMP) Prefetcher; the Stride Prefetcher or Data Prefetch Logic (DPL); and the Next Line Prefetcher or Adjacent Cache Line (ACL) [13, 19–21]. Figure 1 shows the AMP prefetcher captures unique and complex memory access patterns; for example, if cache line 0×0000, 0×0002, 0×0003 has been accessed and then 0×0000 and 0×0002 have accessed again, AMP prefetcher will detect this pattern and fetch 0×0003 cache line. The stride prefetcher captures streaming memory requests, separated by a specific stride length; for example, if cache line 0×0000, 0×0002 has been accessed then DPL fetches 0×0004 and 0×0006 cache lines. The ACL prefetcher is highly active processor. It fetches the next cache line adjacent to the recent memory accesses; for example, if 0×0003 has been accessed, then ACL detects and fetch 0×0004 cache line.

2.2 Prefetcher aggressiveness

One method for optimizing memory access latency is prefetching. Prefetchers request memory lines that may be necessary in the future, a more aggressive prefetcher will pull in more memory. Prefetching aggressiveness is controlled by two parameters: prefetcher distance and prefetcher degree.

Prefetch distance defines how far ahead the prefetcher will request memory ahead of a recently used cache line. Distance corresponds to the time the prefetcher stays ahead of the access stream. Longer distances are beneficial during predictable workloads; however, sporadic workloads will suffer from prefetchers with aggressive distance.

The prefetcher degree determines the number of lines prefetched in a single request. A high degree prefetcher fetches many cache lines around a recent cache access whereas a conservative prefetcher requests a few lines around the latest memory access [10].

Static aggressiveness levels are employed in current systems in the DPL prefetcher; the APL aggressiveness only changes when its accuracy is high. [22]. We believe there is opportunity to optimize prefetch performance by dynamically changing prefetcher aggressiveness degree via a smart manager. The work presented in this paper manages the prefetcher aggressiveness degree.

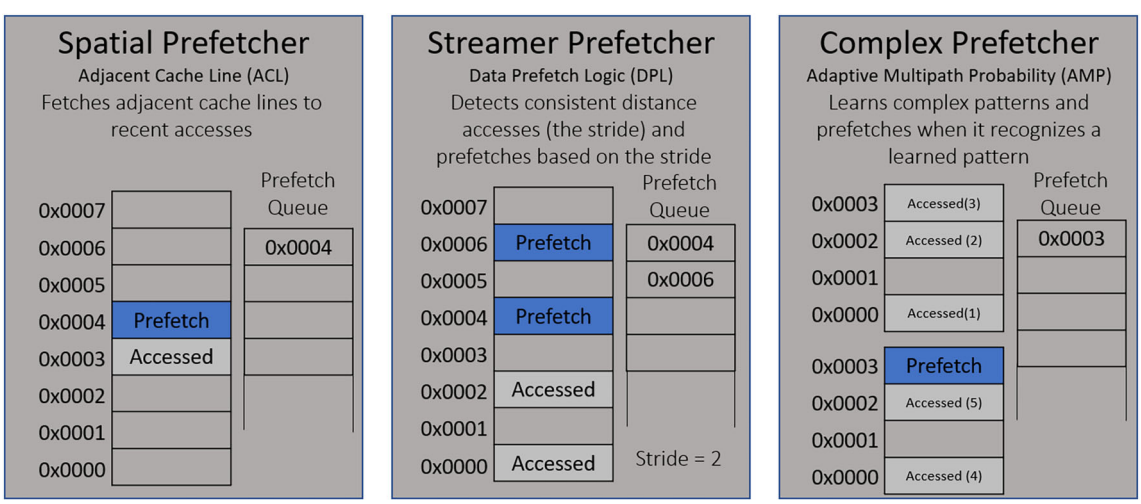

Fig. 1 MLC prefetchers for Intel Xeon

3 Related work

Currently, the DPL prefetcher operates with a static aggressiveness while the ACL prefetcher increases aggressiveness when its accuracy is high (when the memory it prefetches is used by the program) [19]. We believe managing the aggressiveness of multiple prefetchers at runtime must be investigated. Several prior works details managing prefetcher aggressiveness and its effects [9, 13–18]. Ebrahim et al. proposes HPAC, which dynamically changes aggressiveness based on telemetry data from local core and global system perspectives. Panda et al. focuses on assigning aggressiveness at runtime. Panda built on a single core prefetcher aggressiveness manager presented in CAFFINE [17], by proposing SPAC: A Synergistic Prefetcher Aggressiveness Controller for Multi-Core Systems [18]. SPAC executes in two stages: exploration and implementation. The exploration stage is when the meta-controller explores combinations of aggressiveness levels. During implementation, the aggressiveness configuration is set by the SPAC agent using the optimal aggressiveness configuration found during the exploration phase. Panda compared the performance of SPAC and HPAC, demonstrating SPAC had 12.3% improvement over HPAC, and 21% over no prefetching for a 4-core system. However, our previous paper [2] ADM shows 25% improvement over no-prefetching on a single core system. SPAC is the highest performing solution to our knowledge which manages prefetcher aggressiveness similarly to our solution [2]. Since panda results showed performance improvement on 4-cores system, SPM hierarchical model applied to 4-cores system should show even higher performance benefits compared to prefetching turned off.

4 Smart prefetchers manager architecture

This paper introduces Smart Prefetchers Manager (SPM) which is a framework employing reinforcement learning to find the optimal prefetcher aggressiveness policy for multiple prefetchers at runtime. Our goal is to make multiple data prefetchers cooperate in a constructive way to improve the instructions per cycle (IPC). SPM manages the L2 prefetchers to optimize the global IPC at run time by learning the optimal configuration for each workload phase, using machine learning algorithms to optimize the global system performance. SPM encourages synergism between the L2 prefetchers by distributing high-impact telemetry data between prefetchers, which is used to intelligently adjust hardware parameters to improve performance globally.

SPM has two levels of intelligent agents: the high-level SPM and the lower level L2 prefetcher agents. The prefetcher agents at the L2 cache minimize cache misses for each prefetcher and learn the optimal aggressiveness. The high-level SPM agent learns to optimize instructions per cycle (IPC) by controlling low-level agents. Figure 2 shows SPM architecture that is targeting three L2 prefetchers: AMP, DPL, and ACL. Each of these prefetchers uses a reinforcement learning algorithm to make an intelligent decision based on previously learned knowledge and exploration to reduce the cache misses and optimizes the IPC.

The SPM manages the L2 prefetchers aggressiveness to optimize the global IPC at run time by identifying the states and actions and rewarding the L2 prefetchers with reward value based on each prefetcher impact to the overall IPC. It interacts with the L2 prefetchers by requesting the current state aggressiveness of the prefetcher. Then, the agent tells SPM what action it will take. Possible actions for the low-level agents are to increase aggressiveness, decrease aggressiveness, or remain the same. If the action is beneficial then the SPM provides the agent a large reward, otherwise, the agent will receive a small reward.

The machine learning algorithm at the manager level optimizes the global performance of the system and the lower-level agent optimizes local goals.

Q-learning at the manager level supports three states; high, medium, and low which identifies groups of aggressiveness degrees. It has three different actions: increase, decrease, or maintain the aggressiveness degree. SPM calculates the reward value that is given to the prefetchers agent for each state action based on the IPC or PPW or power.

5 Methodology

This section presents three techniques that are applicable to SPM architecture. It includes investigating the MLC feature interactions by applying prefetchers toggling, and showing the importance of reinforcement learning technique, in addition to the offline and online techniques.

5.1 MLC prefetchers toggling configurations

To understand how different prefetchers interact with each other, this section presents an investigation of the interactions of the MLC prefetchers. Current systems and specifically baseline configurations enable the ACL, DPL, and AMP prefetchers by default. Ideally, this triple prefetcher configuration should perform better than a single or dual prefetcher configuration. A preliminary investigation was showing the performance impact of enabling or

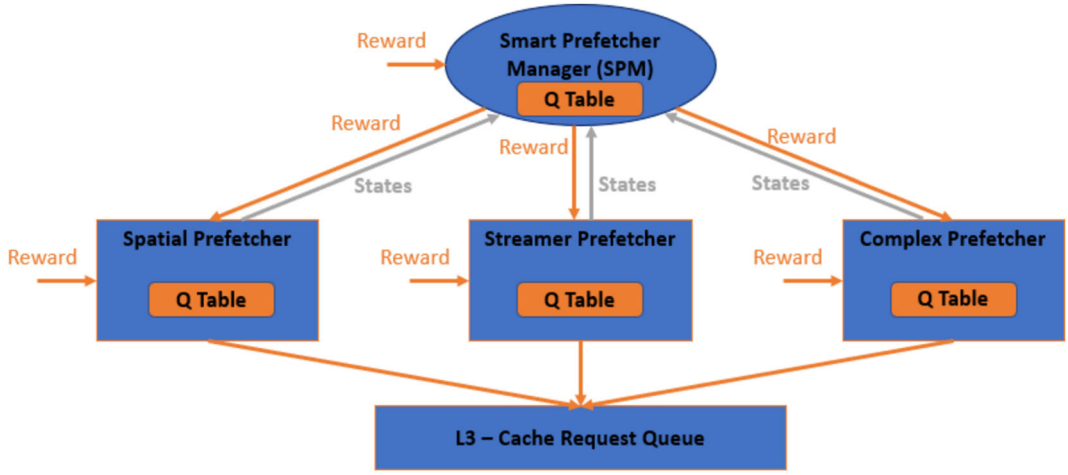

Fig. 2 SPM architecture

disabling the prefetchers, understanding the interaction of the three prefetchers and motivating future work for prefetcher synergism.

Figure 3 shows our experiments to understand the MLC features interactions. Multiple configurations were compared: a system with ACL, DPL, AMP prefetchers off, a system with ACL, DPL, AMP are on (baseline), a system with only single prefetcher enabled, a system with dual prefetchers enabled. Each system configuration's evaluation used traces of regions of interest within workloads from the SPEC CPU2017 suite. Simulation is used to model the system configuration and the performance metric for comparison was IPC to understand different configurations performance impact. The results from the simulation are shown in Fig. 3.

Based on the results, only 50% of traces benefited from all on configuration (baseline) and 26% of traces benefited from all off configuration, 24% shows no change to the performance. As you see in the other 6 configurations, different configurations can constructively or destructively impact the overall performance and most of the configurations show 30%- 50% of the traces track with the default all on configuration.

To summarize all results, Fig. 4 shows that the system is underperforming on 86% of the traces due to negative interactions between the ACL, DPL, and AMP prefetchers. Therefore, a smart prefetcher management scheme should maximize the systems performance.

These preliminary results show enabling and disabling prefetchers within an entire system has significant impact on performance. Motivated by these initial findings and supported by prior art, this investigation focused on optimally configurating prefetchers via a fine-grained solution managing the prefetcher aggressiveness degree. A

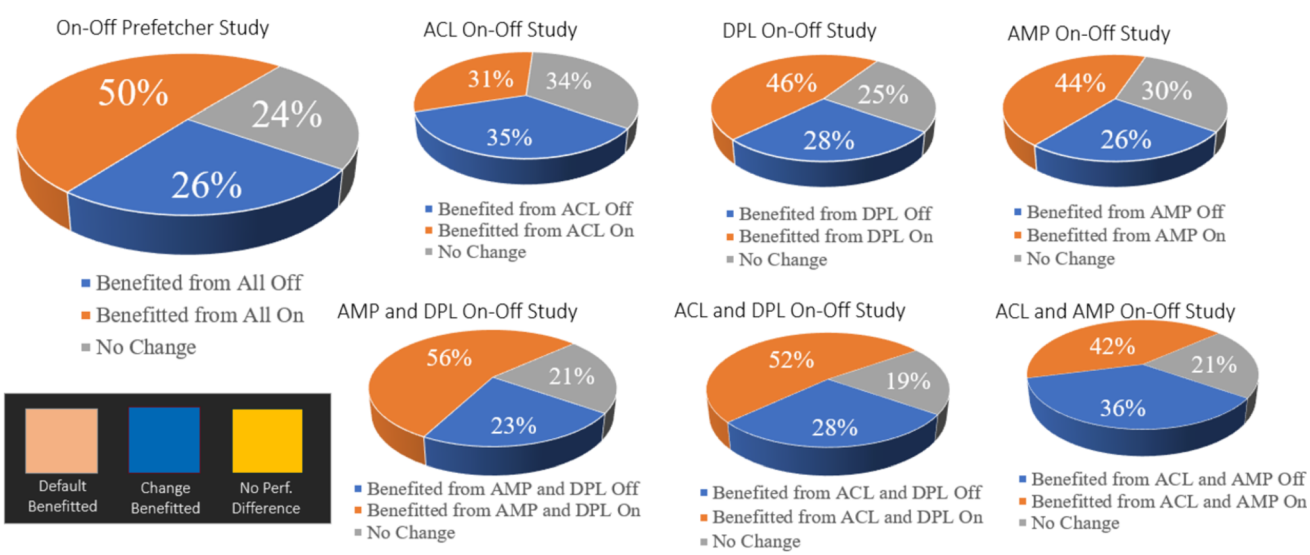

Fig. 3 Workload results of on–off prefetchers configurations

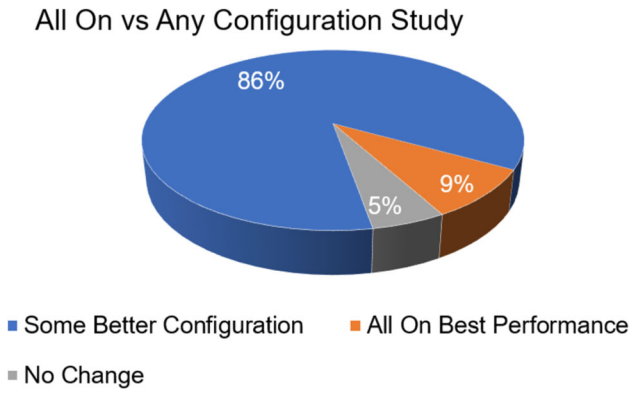

Fig. 4 workload results compared with all configurations

prefetcher with aggressiveness degree zero has the equivalent functionality of disabling a prefetcher, and non-zero aggressiveness degrees have the equivalent functionality of enabling a prefetcher and weighting its performance by an undiscovered scaling factor.

5.2 Reinforcement learning

Reinforcement learning algorithms are a form of dynamic programing requiring minimal computational and memory demands [22]. Q-learning is a reinforcement learning algorithm that is used for an online task. It is unsupervised and modeless. It is used to learn a policy that maximizes total rewards. A Q-learning agent is composed of states, actions, and rewards. An agent represents the world it exists in using states noted as "s". At every update period, an agent can take an action "a" of a set of actions "A" given its current states. The agent receives a reward "r" for action "a", taken in states. If the action is beneficial, then the agent receives a large reward. If the action is not beneficial, the agent receives a small reward. [2].

At each state, a Q-learning agent must learn the reward values by taking an action and receiving a reward. To do that, the agent first identifies the current state by surveying its environment to decide its state. Then, the agent chooses an action "a" that maximizes the reward "r" in states, notated $\max(Q(s,A))$. This agent may decide with some probability ε to take a random action allowing it to explore. After the agent decides which action to take, the agent receives a reward. This reward is based on the action that was taken, if the action was beneficial to the environment, then the agent receives high reward, otherwise the agent receives small reward.

The value $Q(s,a)$ is updated using the Bellman equation shown in Eq. 1.

$$Q(s,a) = Q(s,a) + \alpha(r + \gamma * \max(Q(s',A) - Q(s,a)) \tag{1}$$

The Bellman equation is composed of two parts: the previously learned knowledge with reward $Q(s,a)$, and the update using the actual reward r [23]. The learning rate α weights the received reward r against the assumed reward $Q(s,A)$. Note that this equation uses the best reward from the new state in the reward for the previous action. This is useful because it weights future rewards in the agents' current reward. The discount factor γ scales the effect of the future reward on the previous state-action pair.

5.3 Offline and online learning

This method targets building aggressiveness configurations by running each trace with all possible aggressiveness knobs and picking the knobs that deliver the highest IPC to build the best configuration for a specific trace. For this technique we come up with the best run configuration offline and feed it back to certain traces at runtime. For example, SPM calculates IPC every 5000 cycles and decides which aggressiveness delivered the highest IPC for that specific trace to build a "Best Run" using the samples with highest IPC. With this method, building the knowledge offline can further optimize to the best configuration at runtime faster and expect a higher IPC optimization. We have implemented and evaluated this approach with highly sensitive prefetchers to show the effectiveness of our approach as explained in the results section.

In addition, analyzing workload traces shows that most of the workloads are running in distinct modes. For example, Fig. 5 shows one workload running without SPM enabled. This workload contains multiple phases which makes it harder to learn and optimize for IPC over time. However, if the reinforcement learning learns each phase, the classifier could point to one of a set of Q-tables in each agent. When the mode changes, the previous learning is saved, and a fresh Q-table will learn the new mode to continue to optimize the IPC.

6 SPM implementation

This section presents the implementation of Smart Prefetchers Manager (SPM) by using the general Q-learning algorithm and proposing a state, action, and reward scheme to best manage the prefetcher aggressiveness degree on the three MLC prefetchers. In this section, we

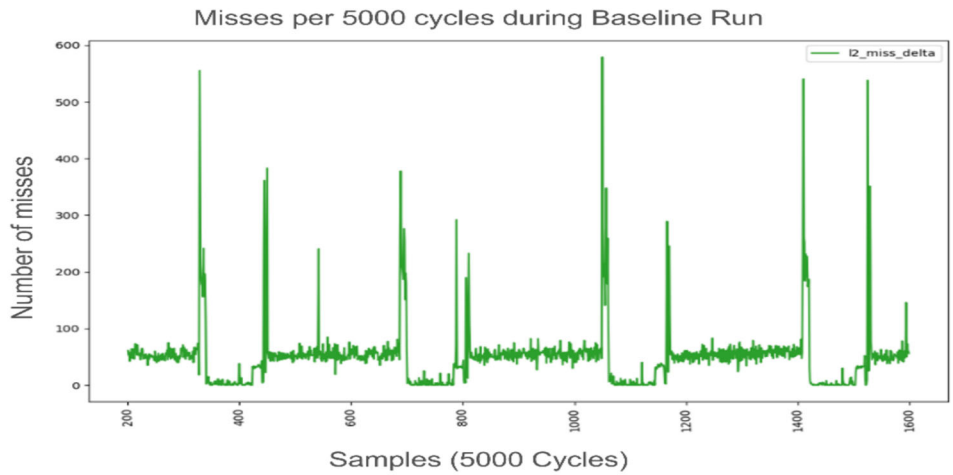

Fig. 5 Example of workload's behavior

describe the SPM initialization and configuration in addition to calculating rewards and the impact on the overall performance.

6.1 SPM initialization and configuration

The Smart Prefetchers Manager (SPM) is a hierarchal model designed to maximize the system performance such as IPC. It uses Q-learning to manage the prefetchers aggressiveness with rewards based on the impact on the IPC. The Q-learning table includes states and actions with each state representing an aggressiveness degree and the action would be increase, decrease, or maintain the same aggressiveness degrees. For example, an SPM initialized with 5 states, $< 0,1,2,3,4 >$, those states represent that the agent may learn a policy to optimize the IPC using disabled mode, a single line prefetch mode, dual line prefetch mode, triple line prefetch mode, or quadruple line prefetch mode.

Table 1 shows the possible states, actions, and rewards learned by the SPM which is the high-level agent. At the beginning of a new program, the table is initialized with all zeros and gets updated each time an action is taken, after a reward is provided. The anticipated reward values are shown for each state-action pair. For example, the reward for state 0, action "increase", the reward is 0.50. The agent determines the next action to take by identifying the action with the maximum assumed reward given the current state. The low-level agent can then accept the action. This manager compares five states and actions. As we add more states, we expand this Q-table and cover more granular configuration and more states.

The concept of SPM is that it learns over time. At the beginning, the Q-table is initialized with all zeroes and the rewards are not calculated yet. The agent learns a lot and makes more frequent random actions, then it learns less and makes fewer random actions. For example, SPM might start at state 0, take a random action and calculate the IPC, then take another random action and calculate the IPC so that the SPM has moved to a different state as it learns. Once SPM learns the environment it converges to the optimal configuration and optimizes the IPC. It usually spent the first 25% of the trace learning and making random action to learn the optimal configuration, once it learns the environment, then it starts to optimize for IPC. Occasionally, in case of the environment changes and states reward needing to be changed, the agent will take a random action and leave the previously optimal configuration. However, when the agent is in a steady state with no random actions required it converges optimally to a state similar to Table 1 and delivers the performance improvement at runtime.

Table 1 Example of Q-Table in SPM Agent

State	Actions		
Degree	*Decrease*	*Same*	*Increase*
0	1.50	1.00	0.50
1	0.50	1.00	2.00
2	1.00	2.00	0.50
3	2.00	1.50	2.00
4	3.00	2.00	1.00

6.2 SPM algorithm and calculating rewards

In this architecture, one of the critical decisions for Q-learning algorithm is when to update its rewards. When analyzing the prefetcher cache line impact on the performance, we noticed that occasionally the prefetcher configuration doesn't show an immediate impact on the IPC and the prefetched cache line is used after thousands of instructions. To analyze the changes of IPC and optimize the MLC prefetchers, each agent is updated after executing many instructions. Those instructions are loads or stores. For our experiments, as it is explained in Sect. 7, the total number of instructions was 128,000 instructions. $\sim$ 33% of all instructions are either load or store instructions for the SPEC CPU2017 suite [24]. Therefore, Q-table update can provide enough data to realize and represent the performance impact of the prefetcher against 40,000 load and stores instructions. At every update, IPC is calculated using counters from the associated MLC prefetcher core.

Algorithm 1 shows Q-learning implementation for the initialization and training functions for an SPM. The SPM agent updates its state based on the agent's actions and how it affects the SPM's goal which in our case is the IPC for the program. The SPM agent uses the Bellman equation where the learning, random, and discount rates are configurable, such as, constant increments from low to high, differential, and sinewave. The reward is based on the change in IPC during the sampling period. Then, the SPM decides on the next action based on the best available state. The state within the Q-table conveys how aggressive each of the underlying agents should work towards their goals. In this algorithm, each agent is one entity and gets the same aggressiveness. The intuition of using IPC for the reward was based on optimizing the overall program execution performance. There are many variables that can be tuned such as, the reward, the knobs, the learning rate, random exploration, discount factor. This algorithm allows you to adjust these values to better learn which is better for various workloads. This algorithm can change in the future work to get feedback from the system to learn how to adjust these parameters depending on the workload that is being run.

Algorithm 1 Q-Learning Initialization and Training

```
update_frequency = UPDATE_FREQUENCY
action = 0

class QLearning:
states = []
actions = []
qtable = {}
    function addState(state):
    self.states.append(state)
    function addAction(actions):
    self.actions = a
    function initialize():
    for s in self.states:
        self.qtable[s] = {}
        for a in self.actions:
            self.qtable[s][a] = 0
    function getAction(s):
    max_reward = 0
    for a in self.qtable[s]:
        if self.qtable[s][a] > max_reward then:
            best_action = a
            max_reward = self.qtable[s][a]
    self.old_state, self.old_action = s, a
    return best_action
    function updateReward(r):
  old_reward =
self.qtable[self.old_state][self.old_action]
  self.qtable[self.old_state][self.old_action] +=
                LR * (r - old_reward +
G*future_reward)

function InitializeAgent(MAX_DEGREE):
q_table = new QLearning()
for (i = 0; i < MAX_DEGREE; i++):
    q_table.addState(i)
q_table.addActions(<-1, 0, +1>)
q_table.initialize()

function TrainAgent(q_table,IPC)
    reward = IPC
    q_table.updateReward(reward)
```

7 Experimental results and discussion

In this section, simulation model and tools have been discussed, then the choice of workloads used to evaluate SPM model is explained. Finally, SPM results are conducted and analyzed to show the impact on the performance [25].

We have implemented SPM methodology using high speed multi-core simulator, a next generation × 86 simulator called sniper [26]. It is an interval core model that allows the generation of CPI stacks which shows the number of cycles lost due to different characteristics of the system. This leads to better understanding of the components impact on the overall IPC and system performance. Sniper has an average performance prediction error within 25% at simulation speeds of several MIPS [26].

In addition to that, we decided to use sniper simulator since it has high accuracy which allow us to make architectural comparisons using simulation results. When observing performance increase in Sniper simulator due to architectural change, this will scale proportionally to real hardware. Using this simulation, we have compared different aggressiveness degree configurations. The output results file provides information for the entire system and contains performance counters for each of the major hardware components and that allow the quantification and visualization of the results.

The Sniper simulator was configured using future Xeon 2025 next generation configurations on single core. Technically, software takes a long time in simulating a program's execution. One feature of Sniper is that it can "Fast Forward" through uninteresting regions of code.

Then Sniper can enter "Detailed" mode where it captures all performance statistics within that region. For these studies, this sampling technique was applied to generate results from multiple regions of interest in each workload, capturing specific behaviors.

We have evaluated our approach using various HPC, Cloud and SPEC CPU2017 suite traces of workloads. Those are 532 traces across more than 20 workloads. The traces contain next-generation, industry-standardized, CPU intensive workloads for measuring and comparing compute intensive performance, stressing a system's processor, memory subsystem and compiler. In our previous publication [2] we implemented smart agents for 2 prefetchers, only ACL and DPL, to manage the prefetcher aggressiveness degree using CPU suite only. In this implementation we have implemented a hierarchical smart agent using various HPC, Cloud and CPU suites where we enabled the smart agents at the three MLC prefetchers and those prefetchers are managed by a higher-level manager (SPM) to further optimize the IPC.

To understand experimental setup and implement an SPM hierarchical model with Q-learning agents for the MLC prefetchers, the three smart prefetchers agents ACL, DPL, and AMP were enabled. Each prefetcher with its own Q-learning algorithm to find the optimal prefetcher aggressiveness configuration at runtime. Each of the smart prefetcher agents used the current aggressiveness degree as the state variables. The actions were to increase the aggressiveness, to decrease the aggressiveness, or to not change the aggressiveness. For example: the DPL aggressiveness degree varied from zero to eight lines, and the ACL aggressiveness degree varied from zero to six lines. This means that if the DPL prefetcher detects a stride, it may prefetch up to eight stride-separated cache-lines. Similarly, the ACL prefetcher may prefetch up to six adjacent cache-lines in a single request.

Prefetcher's aggressiveness was updated on intervals of 10,000 instructions. We have tried 5000 intervals, but we found this was not giving enough time to learn and optimize. Therefore, we decided to have an updated interval of 10,000 instructions this period provides ample time for the prefetcher configuration to impact the performance. At each update, the Q-learning agent will take a random action eight percent of the time, and an action predicting the maximum reward ninety-two percent of the time. The reward function for each action taken uses a weighted moving average of the IPC.

SPM evaluates the feedback from the prefetchers to SPM to mitigate the disadvantageous distribution of SPM rewards to prefetchers agents. We do that by having SPM observe L2 smart blocks prefetcher reward history and provide feedback of a custom reward to the L2 prefetcher agents. Our prior publication [2], dual prefetchers had one directional movement of data down the hierarchy where reward distributed equally to both agents unfairly punishing the highly performing prefetcher. For this publication, we are enabling bidirectional communication between the manager and smart level agents to provide more granular feedback. For example: when the SPM agent provides its state to the prefetcher agents, the state is based on a higher level QoS which affect individual prefetcher. If the QoS decreases due to one prefetcher's actions while the second and third prefetchers are improving performance, the SPM would distribute a negative reward to the unperforming prefetcher agents and distributing a positive reward to the well performing prefetchers. This happens when only the first prefetcher is the causal agent.

Hyperparameters, such as DPL, ACL, and AMP degree ranges, update interval, percent chance of a random action, and weights for the weighted moving average were derived using a grid search. In addition, the aggressiveness-degree tuner was evaluated on various workloads from the HPC,

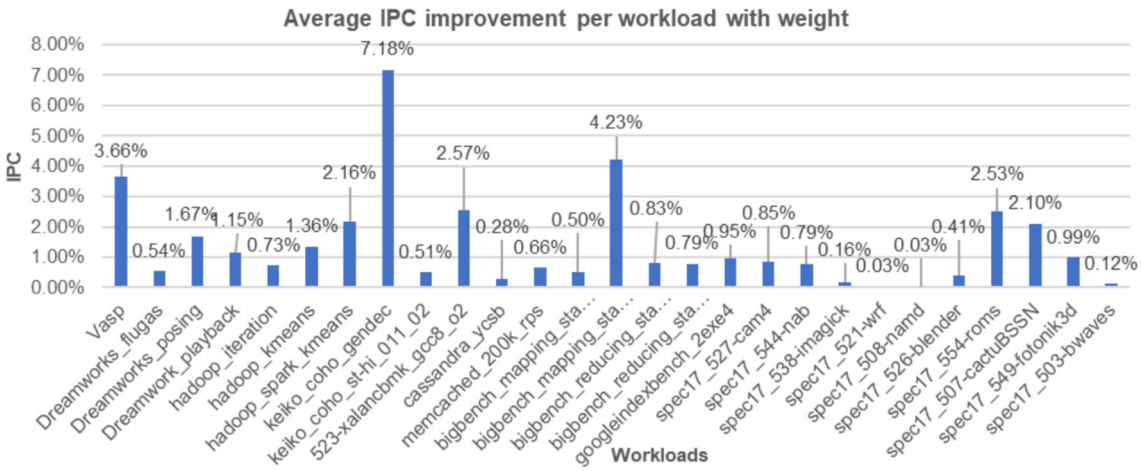

Fig. 6 IPC improvement over baseline

Cloud and SPEC CPU2017 suite exhibiting prefetch sensitivity.

The SPM agent converges to the optimal global aggressiveness degree configuration for each of the three prefetchers, and in some cases found a higher performing configuration than the static aggressiveness degrees. Each workload based on its optimal static aggressiveness degree was classified. Based on the results, Fig. 6 shows that SPM was able to converge to the optimal configuration and optimizes IPC up to 7.18% over baseline. This approach shows an average of 1.4% IPC improvement over baseline configuration as a geomean across all traces of HPC, Cloud and CPU Suite traces (532 traces) that we have evaluated as shown in Fig. 6. SPM agents were able to learn the optimal configuration and converges on runtime to optimize for the IPC over Xeon next generation configurations as baseline for our study. In addition to that, we have analyzed the area impact on the current architecture. Smart Blocks is small agent similar to a state machine so the area and power will be negligible.

We have applied SPM architecture methodology with Xeon next gen architecture to analyze behavior impacts with DRAM latency increase. We have doubled the DRAM latency and used the same set of workloads that are single core, single thread workloads in cycle accurate simulator. Figure 7 shows results of this approach and that shifting memory prefetchers by adding randomness in the run have benefitted from the latency increase and SPM was able to handle the latency increase and continue to optimize and make things better as its able to optimize up to 11.3% IPC improvement over baseline. However, a few workloads showed IPC degradation over baseline. We analyzed those

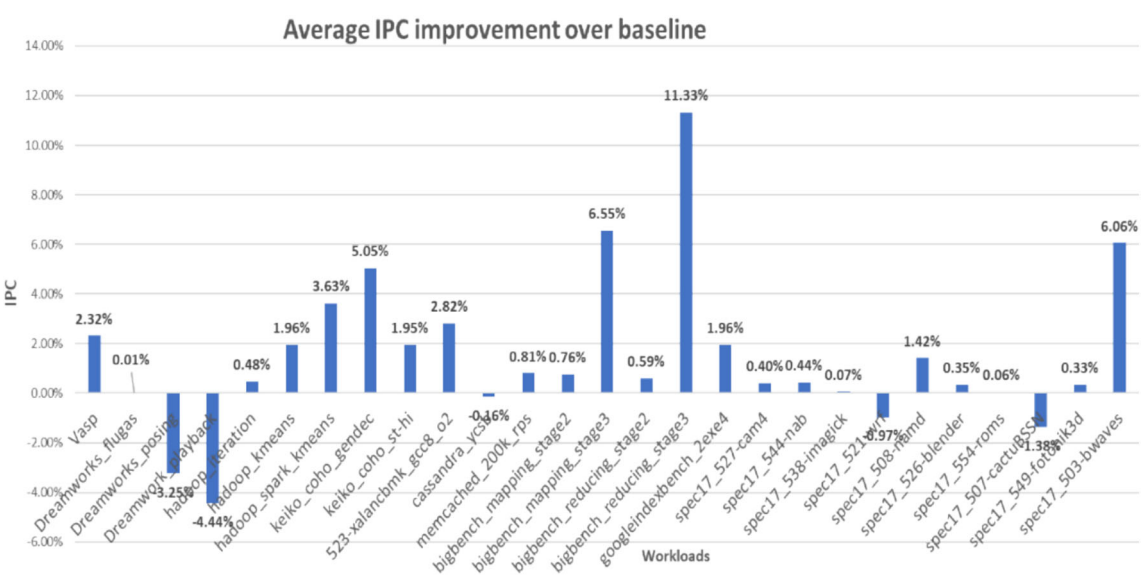

Fig. 7 IPC improvement with doubled DRAM latency compared to baseline

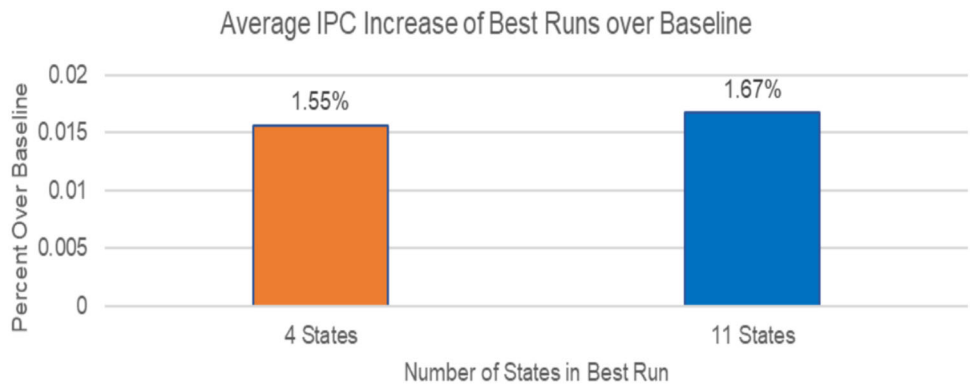

Fig. 8 IPC improvement over baseline using offline and online learning

workloads and noticed that they have high sensitivity to latency.

In addition, we have evaluated the offline and online technique using 523 traces of HPC, SPEC CPU2017, and cloud workloads. This technique has been implemented on cycle accurate simulator and tested using SPM by feeding back the best configuration into specific traces at runtime. We have enabled two aggressiveness Q-table sizes (4 states and 11 states). The minimum number of states the prefetcher can control is 4 and the highest number of states is 11 as we leveraged this range from the Xeon product team to enhance the experiments. Based on the results, Fig. 8 shows that enabling 4 states resulted into 1.55% IPC improvement over baseline. However, enabling more granular architecture with 11 states resulted into higher performance of 1.67% IPC improvement over baseline.

Enabling 11 states improved the IPC for some of the workloads compared to baseline, however, 11 states results showed performance degradation for most of the workloads compared to baseline. The reason behind this degradation is that enabling 11 states requires more actions and more learning time and the overall traces are not long enough to give the ML the time to learn the optimal configurations and converge positively compared to baseline.

We have also enabled various techniques within the Bellman equation to further optimize the IPC. One technique is making the learning rate configurable when the reward at the SPM manager agent decreases significantly. For example, if the reward values are generating high IPC, then the learning rate decreases, however, with lower IPC, we increase the learning rate. This technique is implemented, and it benefited most of the workloads in general, but it requires more investigation to take full advantage of configuring the learning rate across all traces.

To compare both techniques of reinforcement learning and offline and online learning, results showed that using the reinforcement learning technique to manage the MLC prefetchers was yielding higher IPC optimization compared to the offline and online learning for the listed sets of traces.

8 Conclusion

In this work, we have proposed Smart Prefetchers Manager (SPM), a framework using machine learning capabilities to manage the MLC prefetchers aggressiveness and optimize the instructions per cycle at runtime. SPM provides a hierarchical model; each agent quickly converges to the optimal prefetcher configuration using minimal hardware overhead and latency. Q-learning is used to find the optimal prefetcher aggressiveness policy for multiple MLC prefetchers at runtime.

To evaluate the effectiveness of our approach we have used 532 traces of SPEC CPU2017 suite, HPC, and cloud workloads. The SPM demonstrated up to 7.18% higher speedup than the static hardware configuration using reinforcement learning technique.

We have also collaborated with various product teams to evaluate SPM architecture on Intel next gen Xeon architecture, and we are in the process of prototyping SPM architecture. With the major shift toward AI and the importance of AI in Intel future products, we believe such optimizations help deliver magnitude performance gains. Our experimentation showed that a dynamic prefetcher optimization using reinforcement learning is better than a static configuration. This architecture is a solution to optimizing the prefetcher aggressiveness degree because it only requires a few states and actions, thus requiring minimal hardware.

As a future work, we are planning to apply other techniques to further improve the IPC over baseline and extend this work to system level environment.

Author contributions All authors whose names appear on the submission made substantial contributions to the conception or design of the work; or the acquisition, analysis, or interpretation of data; or the creation of new software used in the work; drafted the work or revised it critically for important intellectual content; approved the version to be published; and agree to be accountable for all aspects of the work in ensuring that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

Funding The authors have not disclosed any funding.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. H. Devarajan, A. Kougkas, H. Zheng, V. Vishwanath and X. -H. Sun, "Stimulus: Accelerate Data Management for Scientific AI applications in HPC," 2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Taormina, 2022
2. M. J. Adiletta, F. Fargo, M. Diamond, J. Adiletta, O. Franza and S. Steely, "A Reinforcement Learning Approach to Optimize Cache Prefetcher Aggressiveness at Run-Time," 2023 Tenth International Conference on Software Defined Systems (SDS), San Antonio, 2023
3. J. Doweck, Inside intel core microarchitecture and smart memory access-White Paper, Intel.
4. Le, H.Q., et al.: IBM POWER6 microarchitecture. IBM J. Res. Dev. **51**, 639–662 (2007)
5. Owen, J., Steinman, M.: Northbridge architecture of AMD's Griffin microprocessor family. IEEE Micro **28**, 10–18 (2008)
6. J. Casazza, First the tick now the tock: Intel microarchitecture (Nehalem) - White Paper, Intel, 2009.
7. E. Ebrahimi, C. J. Lee, O. Mutlu and Y. N. Patt, "Prefetch-aware shared-resource management for multi-core systems," 2011 38th Annual International Symposium on Computer Architecture (ISCA), San Jose, 2011.
8. A. E. Papathanasiou and M. L. Scott. Aggressive prefetching: an idea whose time has come. Hot Topics in Operating Systems, 2005
9. W. Heirman, K. D. Bois, Y. Vandriessche, S. Eyerman, and I. Hur, "Near-side prefetch throttling: Adaptive prefetching for high-performance many-core processors," in Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques, ser. PACT '18. New York: ACM, 2018, pp. 28:1–28:11. [Online]. Available: http://doi.acm.org/https://doi.org/10.1145/3243176.3243181
10. Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N Patt. 2007. Feedback directed prefetching: improving the performance and bandwidth-efficiency of hardware prefetchers. In Proceedings of the International Symposium on High Performance Computer Architecture (HPCA). 63–74.
11. M. Kumar, S. Ali Khan, A. Bhatia, V. Sharma and P. Jain, "A Conceptual introduction of Machine Learning Algorithms," 2023 1st International Conference on Intelligent Computing and Research Trends (ICRT), Roorkee, 2023, pp. 1–7, https://doi.org/10.1109/ICRT57042.2023.10146676.
12. M. S. Kupriyanov, Y. A. Shichkina and S. E. Ilin, "Implementation of Temporal Action Detection Workflow for Video Data Analysis with the Use of Machine Learning Operations," *2024 V International Conference on Neural Networks and Neurotechnologies (NeuroNT)*, Saint Petersburg, Russian Federation, 2024.
13. Shih-wei Liao, Tzu-Han Hung, Donald Nguyen, Chinyen Chou, Chiaheng Tu, and Hucheng Zhou. 2009. Machine Learning-based Prefetch Optimization for Data Center Applications. In Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC '09). 1–10.
14. J. Hiebel et al. Machine learning for fine-grained hardware prefetcher control. Proc. ICPP, p. 3, 2019.
15. Saami Rahman et al. "Maximizing hardware prefetch effectiveness with machine learning". In: High Performance Computing and Communications (HPCC), 2015 IEEE 7th International Symposium on Cyberspace Safety and Security (CSS), 2015 IEEE 12th International Conference on Embedded Software and Systems (ICESS), 2015 IEEE 17th International Conference on. IEEE. 2015, pp. 383–389.
16. A. Aziz, M. Cireno, E. Barros and B. Prado, "Balanced prefetching aggressiveness controller for NoC-based multiprocessor", SBCCI '14 Proceedings of the 27th Symposium on Integrated Circuits and Systems Design, 2014.
17. Panda, B., Balachandran, S.: CAFFEINE: a utility-driven prefetcher aggressiveness engine for multicores. ACM Trans. Archit. Code Optim. **12**, 25 (2015). https://doi.org/10.1145/2806891
18. Panda, B.: SPAC: a synergistic prefetcher aggressiveness controller for multi-core systems. IEEE Trans. Comput. **65**, 3740–3753 (2016)
19. Intel 2016. Intel 64 and IA-32 Architectures Developer's Manual: Volume 3C. Intel
20. Saami Rahman, Martin Burtscher, Ziliang Zong, and Apan Qasem. 2015. Maximizing Hardware Prefetch Effectiveness with Machine Learning. In Proceedings of the 17th International Conference on High Performance Computing and Communications. 383–389.
21. Vish Viswanathan. 2014. Disclosure of H/W Prefetcher Control on some Intel Processors. Technical Report. Intel.
22. Christopher JCH Watkins and Peter Dayan: Q-learning. Machine learning **8**(3–4), 279–292 (1992)
23. Richard Bellman, Dynamic programming, Princeton University Press, Princeton, N. J., 1957. MR 0090477.
24. Ankur Limaye and Tosiron Adegbija. A workload characterization of the spec cpu2017 benchmark suite. In Performance Analysis of Systems and Software (ISPASS), 2018 IEEE International Symposium on, pages 149–158. IEEE, 2018.
25. Arulkumaran, K., Deisenroth, M.P., Brundage, M., Bharath, A.A.: Deep reinforcement learning: a brief survey. IEEE Signal Process. Mag. **34**(6), 26–38 (2017)
26. The Sniper Multi-Core Simulator. (2020). Sniper. Retrieved July 24, 2020, from https://snipersim.org/w/The_Sniper_Multi-Core_Simulator

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.